

Relatório 16 – Redes Neurais Convolucionais 1

Edryck Freitas Nascimento

1. Introdução

O objetivo desta aula foi aprender sobre Redes Neurais Convolucionais e a prática de construir uma rede neural convolucional. A tarefa consistia em assistir as seções 8, 9 e 10 do curso “*Deep Learning com Python de A a Z – O Curso Completo*” e desenvolver algo prático para mostrar os conhecimentos adquiridos. Como prática, criei e treinei um classificador de texturas na histologia do câncer colorretal.

2. Desenvolvimento

- **Obtenção de Dados:** Peguei o *dataset* no catálogo do *TensorFlow*, usei o *histologia_colorretal*, ele é um conjunto de dados de classificação de texturas na histologia do câncer colorretal com cada exemplo sendo uma imagem RGB de 150x150x3 de uma das 8 classes. Ele vem junto da biblioteca *tensorflow-datasets*, sendo necessário instalá-la antes de iniciar o *notebook*.
- **Preparação dos Dados:** No processamento dos dados, precisei realizar a separação dos dados, separar a imagem do rótulo, e também separar em dados para treino e teste, pois ele vem em apenas uma divisão, chamada *train*, com 5.000 dados. Primeiro separei as imagens dos rótulos e usei *train_test_split* para separar em dois conjuntos com separação de 20/80, sendo 20% (1.000) para teste e 80% (4.000) para treino. Normalizei os dados das imagens e transformei os rótulos em dados categóricos.
- **Configuração do Modelo:** Inicialmente montei o mesmo feito em aula, mas após os testes, foi necessário realizar várias mudanças para que se encaixasse no meu contexto para uso, no total fiz cinco *notebooks*. Em todos eles eu usei a função de ativação *ReLU* para as camadas ocultas e a *SoftMax* para a saída, afinal, é um problema de classificação múltipla. Usei o otimizador Adam e sua taxa de aprendizagem padrão (0.001), para o treinamento, usei a função de perda *categorical_crossentropy*. Após falar sobre as configurações do modelo, vou explicar o motivo das mudanças.
 - **Primeiro notebook:** O inicial foi de dois blocos com uma camada convolucional seguida de uma de normalização e uma de *pooling*,

após as duas essas duas camadas, usei uma de *Flatten*, uma densa com *dropout* e por último a da saída com oito neurônios.

- **Segundo notebook:** Reduzi a resolução da imagem de 150x150 para 64x64, adicionei uma camada de *dropout de 0.5* entre o *Flatten* e a primeira camada densa, a primeira camada densa alterei para 64 unidades e aumentei o segundo *dropout* para 0.5. Também aumentei a base de treino usando a classe *ImageDataGenerator*, pois notei que para este treino, a minha base de dados inicial era muito pequena.
- **Terceiro notebook:** Aumentei a resolução das imagens para 128x128, aumentei o filtro da segunda camada convolucional de 32 para 64 e aumentei a quantidade de épocas do treinamento de 10 para 25.
- **Quarto notebook:** Adicionei mais um terceiro bloco com uma camada convolucional com filtro de 96, camada de normalização e uma de *pooling*.
- **Quinto notebook:** Só aumentei a quantidade de épocas de 25 para 100.
- **Treino:** O modelo configurado é alimentado com o conjunto de dados de treino para aprender os padrões e ajustar seus pesos internos.
- **Validação:** Após o treino, o modelo é avaliado usando um conjunto de dados de validação separado. Se a acurácia do treino for alta, mas a da validação cair, sabemos se está ocorrendo um *overfitting* e o modelo não está aprendendo a generalizar.
 - **Primeiro notebook:** No treino, houve uma acurácia de aproximadamente 87%, mas a de validação foi muito baixa, as causas foram por causa de um *overfitting* do modelo, na primeira camada densa teve um total de 4.308.544 parâmetros, o *Flatten* produziu 41.472 valores e eu conectei tudo em apenas 128 neurônios sem nenhuma regularização. Também, apenas 4.000 dados para treinos eram poucos, resultando em muitos parâmetros para poucos dados.
 - **Segundo notebook:** O *dropout*, o *augmentation* e redução do tamanho ajudou, a acurácia subiu para aproximadamente 65% no treino, mas na validação ficou com cerca de 44%, mas dava para melhorar ainda mais.
 - **Terceiro notebook:** Experimentei aumentar as imagens para 128 *pixels*, adicionei mais épocas e aumentei o filtro da segunda

camada convolucional para 64, acurácia do modelo caiu para cerca de 36% e de teste ficou em próximo de 40%.

- **Quarto notebook:** Nesta tentativa, adicionei mais um bloco de camada (convolucional com filtro de 96, normalização e *polling*), adicionei porque o *Flatten* ainda passava 57.600 valores para a densa, então achei que ia ajudar diminuir, e ajudou, notei que a acurácia vinha subindo conforme passava as épocas, a acurácia de treino ficou em 66% e na validação com 57%.
- **Quinto notebook:** Como no *notebook* anterior já teve um resultado melhor, então apenas aumentei a quantidade de épocas para 100, ao final, obtive uma acurácia de 89% no treino e 86% na validação.
- **Carregar Melhor Modelo:** O modelo do quinto notebook foi o que obtive os melhores resultados, como a acurácia estava crescendo continuamente a cada época, apenas aumentei a quantidade de épocas para 250 e obtive os resultados de 94% de acurácia no treino e 83% na validação, calculando outras métricas, obtive:

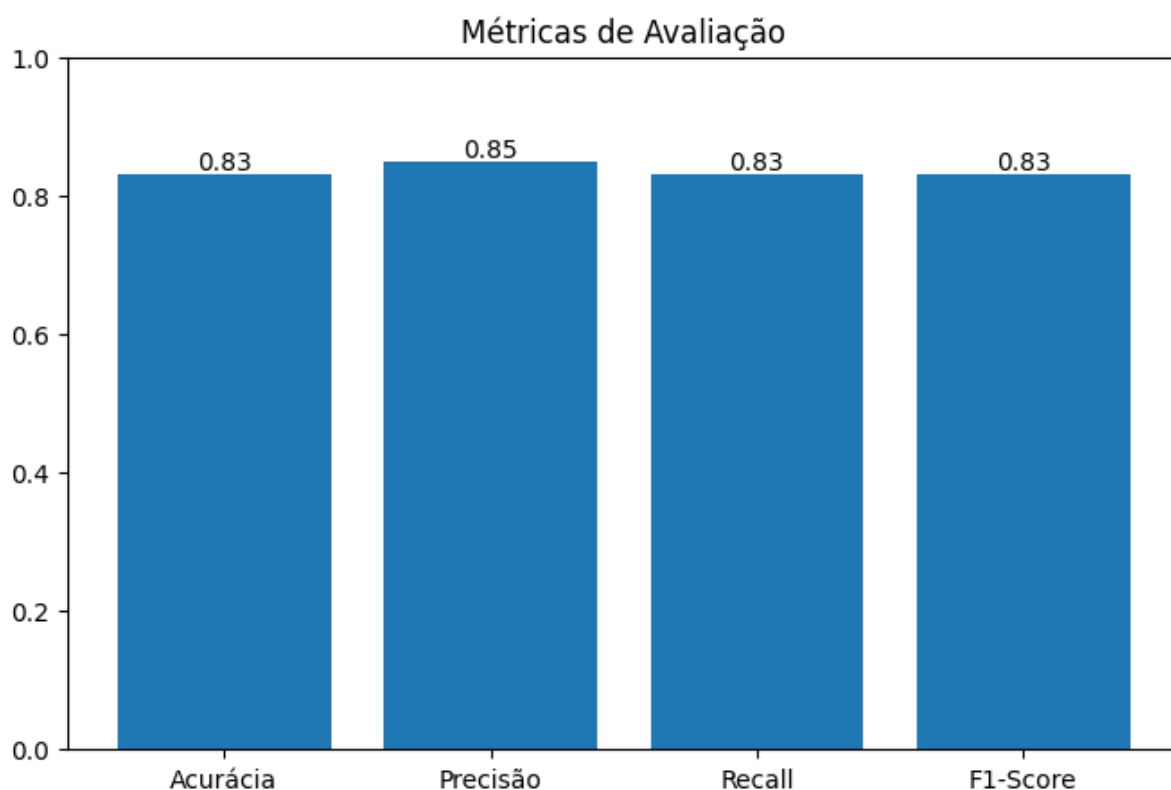


Figura 1. Métricas utilizadas para avaliação. (autoria própria)

Os resultados foram muito bons, a similaridade da previsão com recall mostra que o modelo tem uma precisão de 85% e que ele classificou

corretamente 83% dos dados, tendo uma média harmônica entre precisão e recall de 83%.

A Matriz de Confusão dele também foi impressionante:

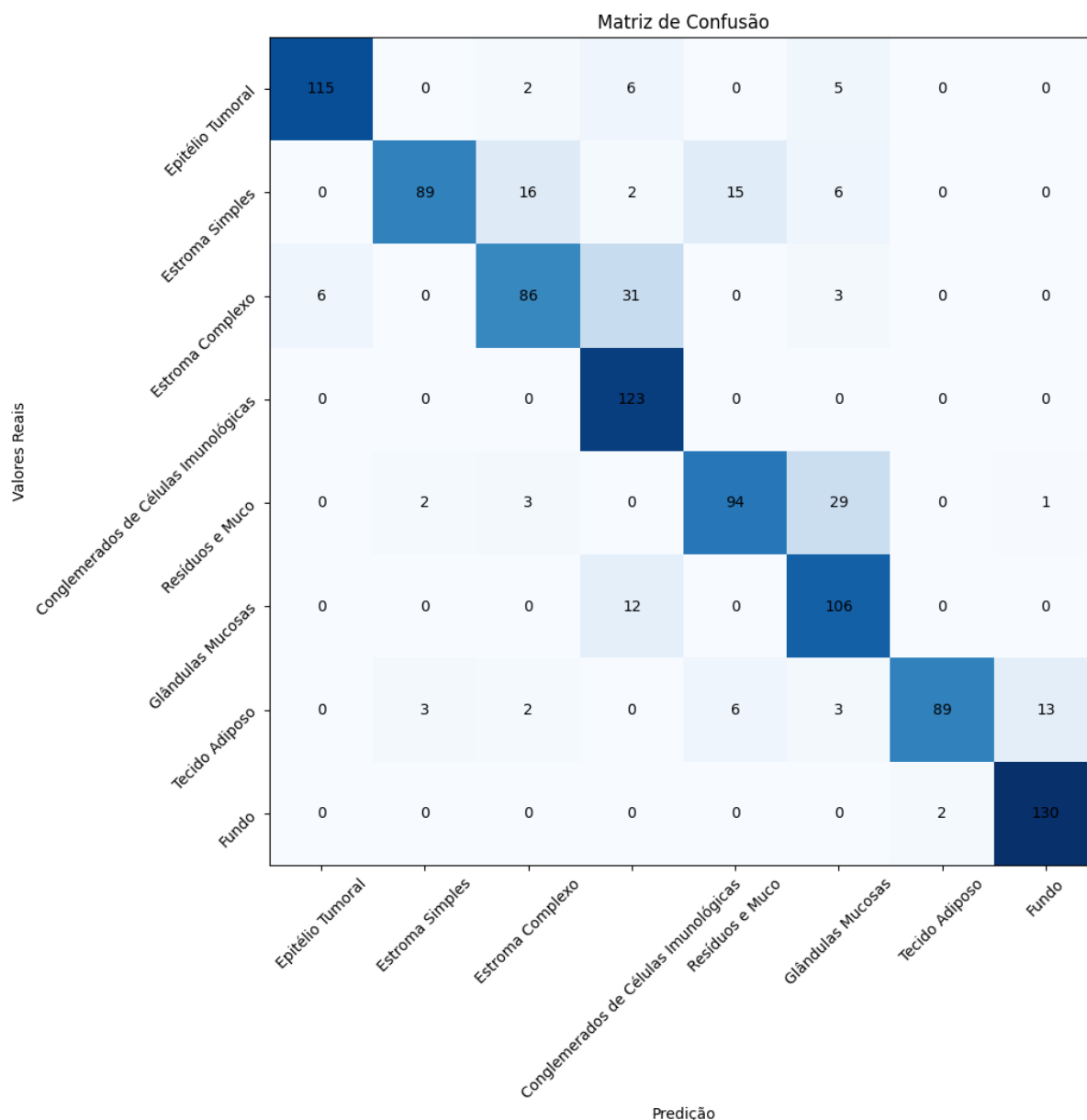


Figura 2. Matriz de Confusão do modelo final. (autoria própria)

O modelo tem a diagonal principal bem-marcada com poucas vezes que teve erros.

Mostrando a confiabilidade dos resultados na curva ROC foi de:

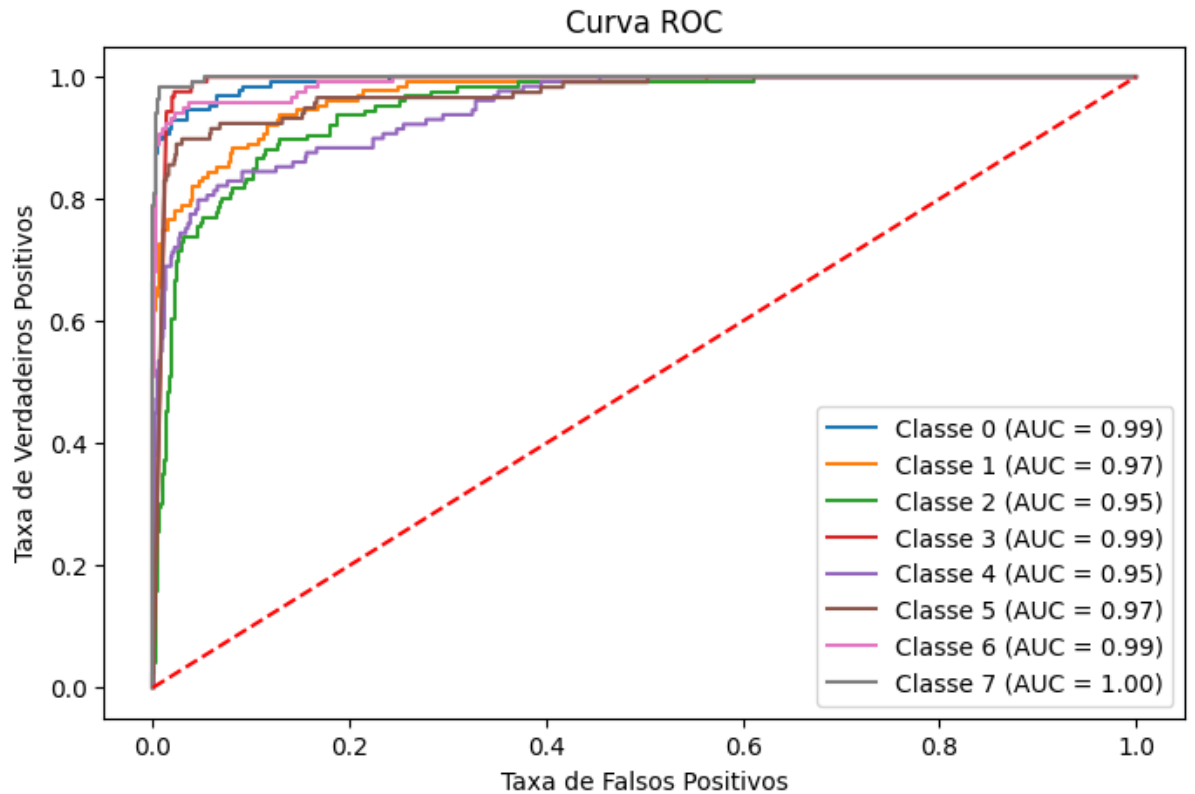


Figura 3. Curva ROC e AUC de cada classe. (autoria própria)

Esse foi meu primeiro modelo que teve um AUC de 1, o que indica perfeito, e mesmo olhando para as outras classes, a menor foi de 0.95 (Classe 2 e 4).

A configuração final que foi utilizada na construção dessa rede neural foi:

Camada	Tipo	Configuração	Shape de Saída
1	InputLayer		(150, 150, 3)
2	Conv2D	32 filtros, kernel 3×3, ReLU	(148, 148, 32)
3	BatchNormalization		(148, 148, 32)
4	MaxPooling2D	pool 2×2	(74, 74, 32)
5	Conv2D	64 filtros, kernel 3×3, ReLU	(72, 72, 64)

6	BatchNormalization		(72, 72, 64)
7	MaxPooling2D	pool 2×2	(36, 36, 64)
8	Conv2D	96 filtros, kernel 3×3, ReLU	(34, 34, 96)
9	BatchNormalization		(34, 34, 96)
10	MaxPooling2D	pool 2×2	(17, 17, 96)
11	Flatten		(27.744,)
12	Dropout	taxa 0.5	(27.744,)
13	Dense	256 neurônios, ReLU	(256,)
14	Dropout	taxa 0.5	(256,)
15	Dense	8 neurônios, Softmax	(8,)

3. Conclusão

A criação do classificador mostrou que a configuração da rede neural é tão importante quanto do treinamento e o *dataset* utilizado, como o primeiro modelo sofrendo *overfitting* por excesso de parâmetros e devido ao baixo volume de dados. O resultado com 83% de acurácia na validação, AUC mínimo de 0.95 e uma classe com AUC perfeito, de 1.00, mostrou-se ser possível criar um classificador confiável para imagens médicas complexas, se a arquitetura e os hiperparâmetros serem ajustados iterativamente. A evolução entre os *notebooks* mostrou que o equilíbrio entre profundidade da rede, regularização com *dropout*, *augmentation* e quantidade de épocas é o que determina a capacidade de generalização do modelo.

4. Referências

Curso: <https://www.udemy.com/course/deep-learning-com-python-az-curso-completo/>.

Dataset: https://www.tensorflow.org/datasets/catalog/colorectal_histology?hl=pt-br.